

Aula 31 - Testes em WebFlux com StepVerifier

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Por que testar fluxos reativos?

- **Desafios:**
 - Assincronicidade dificulta a previsão de comportamentos.
 - Erros podem ser silenciosos em streams não consumidos.
- **Solução:**
 - `StepVerifier` (Project Reactor) para testes declarativos.

2. StepVerifier vs Testes Tradicionais

Critério	<code>StepVerifier</code>	<code>@SpringBootTest</code>
Foco	Teste de Publishers (Mono/Flux)	Teste de integração comple
Controle do Tempo	Avanço manual (<code>virtualTime</code>)	Tempo real
Verificação	Asserções em cada sinal (next, error, complete)	Asserções no resultado fina



3. Principais Métodos

Método	Descrição
<code>expectNext(T)</code>	Verifica o próximo elemento emitido
<code>expectError()</code>	Verifica se um erro foi emitido
<code>expectComplete()</code>	Verifica se o fluxo completou com sucesso
<code>thenCancel()</code>	Cancela a assinatura do fluxo
<code>verifyThenAssertThat()</code>	Combina verificação com asserções adicionais

Parte Prática (3 horas)

Atividade 1: Testando um Service Reativo (1h)

Service sob Teste (ProductService.java)

```
java

@Service
public class ProductService {
    private final ProductRepository repo;

    public Flux<Product> getExpensiveProducts(double minPrice) {
        return repo.findAll()
            .filter(p -> p.getPrice() > minPrice)
            .switchIfEmpty(Mono.error(new NoSuchElementException()));
    }
}
```

Teste com StepVerifier (ProductServiceTest.java)

```
java

@ExtendWith(MockitoExtension.class)
class ProductServiceTest {
    @Mock
    private ProductRepository repo;

    @InjectMocks
    private ProductService service;

    @Test
    void shouldReturnExpensiveProducts() {
        Product p1 = new Product("Notebook", 4500.0);
        Product p2 = new Product("Mouse", 50.0);

        when(repo.findAll()).thenReturn(Flux.just(p1, p2));

        StepVerifier.create(service.getExpensiveProducts(1000.0))
            .expectNext(p1) // Notebook é caro
            .expectComplete()
            .verify();
    }

    @Test
    void shouldErrorWhenNoProductsFound() {
        when(repo.findAll()).thenReturn(Flux.empty());

        StepVerifier.create(service.getExpensiveProducts(1000.0))
```

```

        .expectError(NoSuchElementException.class)
        .verify();
    }
}

```

Atividade 2: Testando um Controller WebFlux (1h)

Controller (ProductController.java)

```

java

@RestController
@RequestMapping("/products")
public class ProductController {
    @GetMapping("/expensive")
    public Flux<Product> getExpensive(@RequestParam double minPrice) {
        return productService.getExpensiveProducts(minPrice);
    }
}

```

Teste com WebClient (ProductControllerTest.java)

```

java

@SpringBootTest
@AutoConfigureWebTestClient
class ProductControllerTest {
    @Autowired
    private WebClient webClient;

    @MockBean
    private ProductService service;

    @Test
    void shouldReturnExpensiveProducts() {
        Product p = new Product("Notebook", 4500.0);
        when(service.getExpensiveProducts(1000.0)).thenReturn(Flux.just(p));

        webClient.get()
            .uri("/products/expensive?minPrice=1000")
            .exchange()
            .expectStatus().isOk()
            .expectBodyList(Product.class).contains(p);
    }
}

```

Atividade 3: Testes com Tempo Virtual (1h)

Service com Atraso (PriceAlertService.java)

java

```
public Mono<Alert> checkPrice(Product product, double maxPrice) {
    return Mono.just(product)
        .delayElement(Duration.ofSeconds(1)) // Simula processamento
        .map(p -> p.getPrice() > maxPrice
            ? new Alert("Preço alto!")
            : new Alert("Preço OK"));
}
```

Teste com VirtualTime (PriceAlertServiceTest.java)

java

```
@Test
void shouldCheckPriceInVirtualTime() {
    // Configura o tempo virtual
    StepVerifier.withVirtualTime(() ->
        service.checkPrice(new Product("Notebook", 4500.0), 1000.0))
        .expectSubscription()
        .thenAwait(Duration.ofSeconds(1)) // Avança o tempo virtual
        .expectNextMatches(alert -> alert.getMessage().contains("alto"))
        .expectComplete()
        .verify();
}
```

Tarefa para Casa

1. Teste de Erro:

- Criar um teste para um `Mono` que falha após 3 segundos.

2. Teste de Backpressure:

- Verificar se um `Flux` respeita a demanda do assinante com `.expectRequested()`.

Próxima Aula

- Tópico: Projeto Final (Parte 1).
- Atividade prática: Implementar e testar endpoints reativos.

Resultado Esperado

- Domínio dos testes reativos com:
 - Verificação de elementos emitidos.
 - Controle de tempo em operações assíncronas.
 - Integração com `WebTestClient`.

plaintext

- ✓ Testes unitários para Mono/Flux
- ✓ Testes de controllers WebFlux
- ✓ Pronto para o projeto final!

<https://projectreactor.io/docs/core/release/reference/images/stepverifier.png> (*Fluxo de teste com StepVerifier*) 🚀